WORKSHOP HANDOUT

Autonomous Agents

OpenClaw & ZeroClaw — Hands-On für Entwickler

OpenClaw & ZeroClaw — Developer Hands-On Guide

🐾

OpenClaw

Node.js · Semantic Memory · Multi-Agent Gateway
Lokal oder in der Cloud

🦀

ZeroClaw

Rust · 3MB Binary · 65 Provider · Hardware-Support
Lokal oder in der Cloud

📅 2026 👤 Oskar · execute008 🌐 openclaw.ai · zeroclawlabs.ai

Was ist ein autonomer Agent?

GRUNDLAGEN / BASICS

Ein **autonomer Agent** ist ein KI-System, das nicht nur antwortet — es *handelt*. Es kann Werkzeuge verwenden, Entscheidungen treffen, Aufgaben sequentiell ausführen und proaktiv auf seine Umgebung reagieren.

*An autonomous agent doesn't just respond — it **acts**. Tools, decisions, sequences — and it can initiate without user input.*

Chatbot vs. Agent

❌ Klassischer Chatbot

- Fragt → bekommt Antwort
- Kein Gedächtnis zwischen Sessions
- Kein Zugriff auf externe Systeme
- Rein reaktiv, immer on-demand

✅ Autonomer Agent

- Empfängt Ziel → plant → führt aus
- Persistentes Gedächtnis
- Tools: Browser, Shell, APIs, Dateien
- Proaktiv — arbeitet ohne User-Input

Die Agent-Schleife / The Agent Loop

👁️ Wahrnehmen (Perceive) → 🧠 Denken (Reason) → 🔧 Handeln (Act (Tools)) → 💾 Merken (Remember) → 🔄 Nächster Turn (Next cycle)

Werkzeuge / Tools

🌐 web_search / web_fetch
Internet-Recherche in Echtzeit

💻 exec / shell
Befehle auf dem Host ausführen

📁 read / write / edit
Dateisystem-Operationen

🌍 browser
Echten Browser steuern (Playwright)

💬 message
Telegram, Discord, Slack senden

🧠 memory_search
Semantische Suche im Gedächtnis

Memory-System

Agenten sind zustandslos — Memory-Dateien sind das Persistenz-Layer:

- `MEMORY.md` — Langzeit-Gedächtnis (kuratiert, semantisch durchsuchbar)
- `memory/YYYY-MM-DD.md` — Tagesnotizen, rohe Logs
- `SOUL.md` — Persönlichkeit & Werte · `USER.md` — Kontext über den Menschen
- `HEARTBEAT.md` — Periodische Aufgaben-Checkliste

> 💡 **Key Insight**
> LLM = Gehirn · Tools = Hände · Memory-Dateien = Langzeitgedächtnis. Erst zusammen entsteht echter Nutzen.

OpenClaw vs ZeroClaw

VERGLEICH / COMPARISON

Beide sind **vollständige Agent-Runtimes** — der Unterschied liegt nicht im Deployment-Ort, sondern im **Technologie-Stack**. Beide können lokal *und* in der Cloud laufen.

> ⚠️ **Wichtig / Important**
> OpenClaw läuft nicht nur in der Cloud, ZeroClaw nicht nur lokal.
> **Beide können beides.**
> Der Unterschied: Node.js vs. Rust, ClawHub vs. 65 Provider, Semantic Memory vs. SQLite.

Feature	🐾 OpenClaw (Node.js)	🦀 ZeroClaw (Rust)
Stack	Node.js / TypeScript	Rust — 3MB Binary, kein Runtime nötig
Deployment	✓ **Lokal + Server + Cloud**	✓ **Lokal + Server + Cloud + Android**
AI Provider	~10 Provider (Anthropic, OpenAI, Ollama...)	**65 Provider** — Ollama, LM Studio, vLLM, DeepSeek, xAI...
Lokal (kein API Key)	✓ **mit Ollama**	✓ **Ollama / LM Studio / llama.cpp / vLLM**
Channels	Telegram, Discord, Signal, WhatsApp, Slack, iMessage...	20+ inkl. Nostr, Matrix, IRC, DingTalk, Lark, Reddit, Bluesky
Memory	Markdown + semantische Suche (memory_search)	SQLite (auto-save) + Markdown
Skills	ClawHub Marketplace (npm install)	Built-in + manuell in ~/.zeroclaw/skills/
Multi-Agent	✓ **MQTT, Gateway, subagents, ACP**	✓ **Daemon + WebSocket Gateway**
Hardware	—	✓ **STM32, Raspberry Pi GPIO, USB peripherals**
Safety Controls	Tool policies, approval flows	E-Stop, OTP, max actions/hour, cost/day limit
Migration	—	✓ `zeroclaw migrate openclaw`
Footprint	Node.js + npm	3MB Binary — kein Node, kein Python

Wann was? / Decision Guide

🐾 OpenClaw wenn...

- JS/TS-Ökosystem bevorzugt
- ClawHub Skills nutzen/publizieren
- Multi-Agent mit MQTT + Subagents
- Deep Telegram/Discord integration
- ACP Coding-Agent Workflows

🦀 ZeroClaw wenn...

- Minimaler Footprint wichtig
- Hardware steuern (GPIO, STM32)
- Maximale Provider-Auswahl (65)
- Niche Channels (Nostr, Matrix, IRC...)
- Migration von OpenClaw geplant

ZeroClaw — Deep Dive

ZEROCLAW FEATURES

Installation (eine Zeile)

```
# Interaktiv — guided setup
curl -fsSL https://zeroclawlabs.ai/install.sh | bash

# Non-interaktiv mit Ollama (kein API Key!)
./install.sh --provider ollama --prefer-prebuilt

# Non-interaktiv mit Anthropic
./install.sh --api-key "sk-ant-..." --provider anthropic
```

65 Provider

☁️ Cloud (API Key)

OpenRouter · Anthropic · OpenAI · Gemini · Groq · Mistral · DeepSeek · xAI · Bedrock · Cohere · Perplexity + 40 weitere

🖥️ Lokal (kein Key)

Ollama · LM Studio · llama.cpp · vLLM · SGLang · Claude Code CLI · Gemini CLI · Osaurus

🔧 Custom Endpoints

`custom:<URL>` — OpenAI-kompatibel

`anthropic-custom:<URL>` — Anthropic-kompatibel

Config (`~/.zeroclaw/config.toml`)

```
# Lokal — komplett offline
default_provider = "ollama"
default_model    = "qwen3:8b"

# Cloud — maximale Power
default_provider = "anthropic"
api_key          = "sk-ant-..."
default_model    = "claude-opus-4"
```

Daemon + Safety

🚀 Always-On Daemon

```
zeroclaw daemon
zeroclaw service install  # systemd
zeroclaw status
zeroclaw doctor
```

Gateway + Channels + Heartbeat + Scheduler — alles in einem Prozess

🛡️ Safety Controls

- **E-Stop** — Notfall-Stopp
- **OTP** — Einmal-Passwort
- **max_actions_per_hour**
- **max_cost_per_day** ($)
- **allowed_commands** — Whitelist
- **workspace_only** — Sandbox

Hardware — einzigartig

🔩 OpenClaw hat kein Hardware-Support — ZeroClaw schon

STM32 Microcontroller · Raspberry Pi GPIO · USB device discovery

`zeroclaw hardware` — scan · `zeroclaw peripheral` — manage

→ IoT-Agents, embedded control, physische Welt steuerbar

Migration von OpenClaw

```
zeroclaw migrate openclaw
# Importiert MEMORY.md, SOUL.md, USER.md und alle .md Dateien
```

✨ **Live Demo heute**

ZeroClaw v0.5.6 läuft auf diesem Server mit Ollama/qwen3:8b — komplett lokal, kein API Key. `zeroclaw agent -m "Hello"` → antwortet in ~1s.

Skills & Multi-Agent-Setups

SKILLS / MULTI-AGENT

Skills — Modulare Erweiterungen

Skills sind Ordner mit einer `SKILL.md` — kein Code-Deploy, nur Text-Instruktionen für den Agenten.

```
my-skill/
├── SKILL.md          # Trigger-Bedingungen + Anleitung
├── references/       # API-Specs, Docs
└── scripts/          # Hilfs-Skripte
```

OpenClaw: `npx clawhub install weather` · **ZeroClaw:** `zeroclaw skills install`

🌤️ **weather**
Wetter via wttr.in / Open-Meteo

📝 **notion**
Notion Pages, DBs, Blocks

💻 **coding-agent**
Claude Code / Codex als Subagent

🎤 **whisper-api**
Audio-Transkription

Multi-Agent — Kommunikation

📡 MQTT (OpenClaw)

- Broker: Mosquitto (Podman/Docker)
- Topics: `hive/agent/inbox`
- Push-basiert, event-driven
- Agents auf verschiedenen Maschinen
- Chat-Inject in laufende Session

🔗 WebSocket Gateway (ZeroClaw)

- Daemon öffnet HTTP + WS Server
- `ws://host:port/ws/chat`
- REST API für externe Trigger
- Subagents via sessions_spawn()
- ACP-Protokoll für Coding-Agents

🐉 Reales Setup: Draco (OpenClaw) + ZeroClaw (heute)

Draco läuft als OpenClaw 24/7 auf diesem Server. ZeroClaw läuft parallel mit Ollama lokal. Beide können über MQTT oder HTTP kommunizieren — unterschiedliche Channels, ein Netzwerk.

```
Draco (OpenClaw)  ←── MQTT ──→  ZeroClaw Demo
  Telegram/Discord/Signal         Nostr/Matrix/IRC/Bluesky
```

Design-Prinzipien

- **Separation of concerns** — Jeder Agent hat eine klare Rolle
- **Skepticism** — Agents vertrauen sich gegenseitig nicht blind
- **Async by default** — Push statt Polling
- **Human-in-the-loop** — Für destructive Actions immer Bestätigung

Quickstart & Ressourcen

SETUP / RESOURCES

🐾 OpenClaw

```
npm install -g openclaw
openclaw configure
openclaw gateway start
openclaw status
```

Voraussetzungen

Node.js ≥ 18 · API Key · optional: Telegram Bot Token

🦀 ZeroClaw

```
curl -fsSL \
  https://zeroclawlabs.ai/install.sh \
  | bash
zeroclaw daemon
zeroclaw status
```

Voraussetzungen

Nichts (Rust wird mitinstalliert). Für lokal: Ollama.

Workspace-Struktur (beide Plattformen)

```
~/.openclaw/workspace/    OR    ~/.zeroclaw/workspace/
├── SOUL.md        # Persönlichkeit des Agenten
├── USER.md        # Kontext über den Menschen
├── MEMORY.md      # Langzeitgedächtnis (semantisch durchsuchbar)
├── HEARTBEAT.md   # Periodische Aufgaben-Checkliste
├── AGENTS.md      # Workspace-Konventionen
└── memory/YYYY-MM-DD.md   # Tages-Logs
```

Ressourcen

OPENCLAW DOCS
docs.openclaw.ai

ZEROCLAW
zeroclawlabs.ai

SKILL REGISTRY
clawhub.com

COMMUNITY DISCORD
discord.com/invite/clawd

WORKSHOP REPO
github.com/execute008/autonomous-agents-workshop

ZEROCLAW NPM
npmjs.com/package/zeroclaw

🚀 Take-Away

OpenClaw und ZeroClaw lösen dasselbe Problem auf zwei Arten. Node.js mit ClawHub und ACP vs. Rust mit 65 Providern und Hardware-Support. Beide lokal oder Cloud. Beide mit Ollama für komplette Datenprivacy.

The real shift: you give the LLM hands, memory, and a persistent identity. That's when it stops being a chatbot and starts being an agent.